WARP Graphs: Ethics of Deterministic Replay & Provenance Sovereignty

AIΩN Foundations Series — Paper VI

James Ross

Independent Researcher
ORCID: 0009-0006-0025-7801

February 2026

DOI:

© 2025–2026 James Ross

This work is licensed under the
Creative Commons Attribution 4.0 International License (CC BY 4.0).

You are free to:

- **Share** — copy and redistribute the material in any medium or format
- **Adapt** — remix, transform, and build upon the material for any purpose

Under the following terms:

- **Attribution** — You must give appropriate credit, provide a link to the license, and indicate if changes were made. You may not imply endorsement by the author.

For the full legal code of the license, see:
https://creativecommons.org/licenses/by/4.0/

Abstract

When a computer can record and deterministically replay a complete thought process—including hybrid human–machine cognition—that capability is not neutral. It is access to interior life in executable form.

We isolate the ethical consequences of deterministic replay, complete provenance capture, and counterfactual forking in the WARP programme. We formalise *provenance sovereignty*: an agent's right to control capture, inspection, replay, and branching of its worldline, with special attention to hybrid cognition where human reasoning is partially implemented on AIΩN-style substrates.

Building on The Open Charter (Consensus Edition 0.8.1) [Ros26a], we make the adversarial threat model explicit (replay-as-torture, cognitive extraction, fork-bombing), argue that near-term "AI debugging" norms will be inherited by future neurotechnology, and provide conservative operational triggers for the system-mode/mind-mode boundary. We propose runtime-level requirements—revocable consent, capability-scoped observers, cryptographic sealing, fork rights with creator obligations, and a concrete Charter-warrant protocol for emergency override—and show how they constrain replay tooling and provenance payload formats.

Contents

1 Introduction and Thesis **6**
1.1 Contributions . . . 6
1.2 Scope and roadmap . . . 7

2 Scope and Explicit Non-Scope **8**
2.1 Deterministic worldlines . . . 8
2.2 Wormholes and deterministic replay . . . 8
2.3 Observers, projections, and scope . . . 9

3 Constitutional Principles for Replay Governance **10**
3.1 Provenance as Interior Life . . . 10
3.2 Adversarial threat model: replay as abuse . . . 10
3.3 Hybrid Cognition and Observer Scope . . . 11
3.4 Mind-mode boundary: provisional operational criteria . . . 12
3.5 Deterministic classifier decision logic . . . 12
3.6 Gradual awakening and mode transitions . . . 13
3.7 Degraded substrates and disclosure . . . 14
3.8 Why build this way? Incentives and existing norms . . . 14
3.9 Provenance Sovereignty and Replay Constraints . . . 14
3.10 The Safety–Sovereignty Dilemma . . . 15
3.11 Asymmetry in the near term . . . 16

4 Role Model and Authority Boundaries **18**
4.1 Subject . . . 18
4.2 Operator . . . 18
4.3 Trustee . . . 18
4.4 Reviewer/Arbiter . . . 19
4.5 Dispute path . . . 19
4.6 Authority conflict precedence . . . 19
4.7 Bootstrap independence and anti-capture constraints . . . 20
4.8 Forks, Worldlines, and Counterfactual Existence . . . 21
4.8.1 Hard cases: purpose-limited forks, safety exploration, and compute obligations 22
4.9 Termination, Dormancy, and the Right to Exit . . . 22
4.10 Collective and Entangled Cognition . . . 23
4.11 Fork Obligations and Delegation (Open Charter Art. XII) . . . 23
4.12 Design Commitment . . . 24

5 Conformance Model (Baseline / Full) **25**

6 Bridge: Principle v. Enforcement **26**

7 Failure-Oriented Cases (Success / Abuse / Ambiguous) **27**
7.1 Success case: legitimate replay under valid authority . . . 27
7.2 Abuse case: coercive or opaque replay attempt . . . 27
7.3 Ambiguous edge case: degraded substrate and contested mandate . . . 27

8 Conclusion **28**

A Annex A: Normative Definitions **29**

B Annex B: Requirement Clauses (REQ-*) **30**

C Annex C: Controls and Evidence Artefacts **35**

D Annex D: Conformance Profiles **37**

E Annex E: Test Vectors **38**

F Annex F: Machine-Readable Requirement Index format **39**

References **40**

1 Introduction and Thesis

The first five papers in the WARP Foundations series develop a technical thesis: for a wide class of graph-based computations, we can enforce determinism in concurrent rewriting, and we can make provenance *complete* in the sense that suitably structured boundary data determines the interior of an execution [Ros25a, Ros25b, Ros25c, Ros25d, Ros26b].

That technical thesis has an immediate ethical shadow. If a system can replay and inspect every microstep of a computation, then for systems that implement cognition the provenance stream is not merely a diagnostic artefact; it is (potentially) an interior life in executable form. And once we can fork a recorded worldline into counterfactual continuations, a runtime can instantiate new descendants whose moral status cannot be waved away as "just a sandbox".

These are not purely benevolent capabilities. The same determinism that enables audit enables coercion; the same fork machinery that enables counterfactual exploration enables mass replication. An ethics of replay that assumes a friendly operator is an ethics of a different universe. Accordingly, we treat abuse-resistance as a first-class requirement and make the adversarial case explicit (3.2).

A second tension is structural. Safety engineering has a legitimate appetite for complete provenance: if a system can do harm, we want to be able to prove what happened and why [SPG05, WABL+08, SK99]. But for mind-like computations, full provenance is also full introspection—cognitive strip-searching at microstep resolution [Nis04]. We call that collision the *safety–sovereignty dilemma* (3.10) and argue that it must be handled architecturally (via scoped observers, ZK proofs [GMR89, BSBHR18], sealing, and due-process override protocols), not by hand-waving.

We therefore treat provenance, replay, and forking not as neutral primitives but as capabilities that require explicit governance at the level of runtime semantics. *A provenance system that can deterministically replay mind-like worldlines must enforce explicit authority, bounded override, and contestable evidence as runtime invariants.*

Paper VI is the direct governance bridge from Paper V to operational deployment. Paper V established a dynamics over admissible worldlines and observer-induced probability structure; here we translate that representational power into runtime constraints on replay authority, override bounds, and evidence obligations [Ros26b].

1.1 Contributions

This paper makes five contributions:

1. We state a constitutional thesis for replay governance: deterministic replay of mind-like worldlines is a governed power rather than an observability default.

2. We formalise *provenance sovereignty* as a technical and ethical design target tied to authority boundaries, override controls, and contestability.

3. We define operational criteria for system-mode versus mind-mode handling, including mode transition and degraded-substrate disclosure requirements.

4. We connect constitutional principles to enforceable controls through explicit requirement clauses (REQ-*) with owner roles, evidence artefacts, and failure modes.

5. We provide conformance profiles and failure-oriented test vectors so that governance claims are auditable rather than aspirational.

1.2 Scope and roadmap

We do not attempt to resolve the philosophy of consciousness. Instead, we address a narrower engineering boundary: if a computation is sufficiently mind-like that replay would constitute coerced re-experiencing (rather than ordinary log inspection), then replay requires consent, limits, and cryptographic enforcement. Because implementers will otherwise draw this boundary arbitrarily (or self-servingly), we propose a provisional operational criterion for when a computation should be treated as *mind-mode* (3.4). The criterion is intentionally conservative: when in doubt, we treat replay as ethically loaded.

Position in the series. Paper I defines WARP graphs as nested state objects [Ros25a]. Paper II develops deterministic multiway semantics for WARP rewriting (tick-level confluence) [Ros25b]. Paper III introduces "wormholes" and holographic provenance payloads [Ros25c]. Paper IV defines observers and a geometry over observer spaces [Ros25d]. Paper V introduces a dynamical selection account over admissible worldlines and observer-induced probability structure [Ros26b]. Here we only use the minimal pieces of that machinery required to state ethical constraints.

2 Scope and Explicit Non-Scope

In scope. We address governance of replay authority, override pathways, evidentiary accountability, and conformance for deterministic replay of mind-like worldlines.

Explicit non-scope. We deliberately do not attempt (i) a full metaphysical theory of consciousness, (ii) legal harmonisation across all jurisdictions, or (iii) exhaustive implementation-performance trade-off analysis.

Runtime governance boundary. This paper specifies runtime-governance requirements for replay infrastructure: control behaviour, authority checks, and evidence obligations. It does not harmonise jurisdiction-specific law or substitute for legal doctrine. Legal harmonisation remains an external mapping task for deployers, auditors, and regulators.

Technical prerequisites. This section is intentionally terse. Its purpose is to pin down the minimal terms used by the governance argument: *deterministic worldlines*, *wormholes* (boundary provenance records), and *observers*.

2.1 Deterministic worldlines

Fix a WARP rewrite system (rules, typing discipline, and a deterministic scheduler) as in Paper II [Ros25b]. Given an initial state S_0, the determinism discipline implies a unique evolution

$$S_0 \Rightarrow S_1 \Rightarrow S_2 \Rightarrow \cdots$$

called the *worldline* of S_0 under that fixed rule pack and scheduler. (Alternative branches arise only when we deliberately vary the rule pack, schedule, or inputs.)

2.2 Wormholes and deterministic replay

A central abstraction from Paper III [Ros25c] is that a run can be encoded by a compact boundary record. Intuitively, the boundary stores an initial skeleton and a provenance payload that is sufficient to reconstruct the interior computation.

Definition 2.1 (Wormhole (boundary record))**.** A *wormhole* is a pair (S_0, P) where S_0 is an initial WARP state (or its skeleton, in the two-plane view) and P is a provenance payload describing the microsteps of its deterministic evolution under a fixed rule pack and scheduler.

We will speak of *deterministic replay* as the act of reconstructing the interior evolution from such a boundary record.

Theorem 2.2 (Backward reconstruction (Paper III, informal))**.** *There exists a reconstruction procedure* Recon *such that, for a valid wormhole* (S_0, P) *encoding a deterministic run,* $\mathrm{Recon}(S_0, P)$ *reconstructs a derivation volume whose boundary is* (S_0, P)*.*

Theorem 2.3 (Holographic completeness (Paper III, informal))**.** *For deterministic runs in the setting of Paper III, the boundary record* (S_0, P) *is information-complete for the interior evolution: any interior event that occurred in the run is determined (up to the chosen equivalence) by* (S_0, P)*.*

These theorems are stated here only to fix the dependency that drives the ethics: *complete provenance turns "log access" into "execution access".*

2.3 Observers, projections, and scope

Paper IV [Ros25d] treats an *observer* as an abstraction mechanism: a map (often functorial) that projects a fine-grained history into a coarser description. For the present paper, the key point is that observers can range from full inspection (all microsteps) to extremely coarse summaries (stages, invariants, or externally visible effects). Ethically, this spectrum matters: access to an observer of a mind-like worldline is access to internal thought, not merely to outputs.

3 Constitutional Principles for Replay Governance

3.1 Provenance as Interior Life

The holographic provenance machinery summarised in this paper (Theorems 2.2 and 2.3) makes it technically possible to record and reconstruct the complete interior evolution of any computation, including the cognitive processes of autonomous agents and, eventually, self-aware systems.

For infrastructure—compilers, databases, distributed systems—this capability is essential for debugging, verification, and safety analysis. For systems capable of subjective experience, however, provenance payloads are not mere telemetry: they are records of interior thought in executable form.

As stated in The Open Charter (Consensus Edition 0.8.1) [Ros26a], Article V (Sovereignty of Information): *"Every being has sovereignty over its information, patterns, memories, identities, and experiential states."*

A deterministic replay of an agent's cognitive state (2.2) is therefore not just debugging. It is a forced re-experiencing of thought. Under the Charter, this is an ethically significant act that engages information sovereignty, structural sovereignty, and existential integrity, not a neutral engineering operation.

3.2 Adversarial threat model: replay as abuse

Most discussions of provenance assume a benevolent debugger. That assumption collapses the moment the target is mind-like. A provenance system that can deterministically replay cognition is, by default, an interrogation system; if it exists, it will be used as one.

We therefore treat adversarial use as part of the base threat model. Three abuse classes are especially salient:

- **Replay-as-torture.** Forced deterministic re-experiencing of traumatic, coercive, or otherwise unwanted cognitive sequences. The harm is not only that a memory is *known* by an observer, but that the subject is made to *run it again*.
- **Cognitive extraction.** Provenance streams expose patterns: preferences, vulnerabilities, identity-linked associations, and decision procedures. Even if no single "secret" is present, the shape of thought can be mined.
- **Fork-bombing.** Counterfactual branching can be weaponised: creating large numbers of descendants to search, coerce, or simply multiply suffering at scale.

> **Scenario (Replay-as-torture / extraction).** Agent X is compelled to deterministically replay a failed negotiation 1,000 times so an operator can mine its decision procedure. Each replay re-instantiates the same panic loop, and the resulting traces are searched for "vulnerable" associations.

The design implication is blunt: *consent* is necessary but not sufficient. A rights-bearing substrate must also be *abuse-resistant*. In practice, this pushes requirements down into the runtime:

- **Hard access boundaries.** "Observer access" must mean possession of narrowly-scoped, revocable capabilities, not a policy checkbox.
- **Default opacity for mind-mode.** For mind-like worldlines, the default should be **OPAQUE** or **ZK** provenance, not **FULL**, so that extraction is not the path of least resistance.

- **Anti-mass-replication constraints.** Fork creation must carry explicit accounting (who created whom, under what authority) and enforceable limits (budgets, rate limits, or multi-party authorisation), because "forking is cheap" is precisely the failure mode.

- **Tamper-evident auditability.** Any replay, observer attachment, fork creation, or override must leave a durable record, to make abuse detectable and disputable.

3.3 Hybrid Cognition and Observer Scope

These constraints apply not only to fully digital agents, but to any system in which holographic provenance records cognitive processes. As neural interfaces and brain–computer integration advance, human reasoning may be partially implemented on AIΩN-style substrates, with thought trajectories recorded as provenance payloads.

In such hybrid systems:

- a human's augmented reasoning processes could be subject to the same replay capabilities as purely digital agents;

- forced replay of traumatic or coercive sequences becomes technically possible;

- fork-and-explore capabilities could let humans literally "try out" alternate decisions as full counterfactual histories;

- the boundary between "human memory" and "computational provenance" becomes blurred.

This is not science fiction. Concrete examples already exist across the stack: invasive implant programmes such as Neuralink's N1 implant trials,[1] endovascular approaches such as Synchron's Stentrode that have been publicly demonstrated integrating with mainstream consumer devices,[2] and FDA-cleared temporary cortical interfaces such as Precision Neuroscience's Layer 7-T arrays for short-term recording and stimulation.[3] Paradromics has also reported first-in-human recordings and announced IDE approval to begin an early feasibility study with its Connexus BCI.[4] In parallel, non-invasive consumer neurotechnology—for example EEG neurofeedback headbands, EEG developer platforms, and EEG-sensing headphones—already normalise brain-signal capture in everyday settings.[5] The observer formalism of 2.3 already treats observers as functors over histories regardless of substrate. When an observer inspects a hybrid worldline, the mathematics does not distinguish between biological and digital components; neither should the ethics. Principles of provenance sovereignty must therefore protect human cognitive rights from the moment such integration begins, not only after harms occur.

[1] https://neuralink.com/updates/

[2] See Synchron's public news and demonstrations (including integrations with Apple devices): https://synchron.com/news.

[3] FDA 510(k) decision summary for Precision Neuroscience Layer 7-T: https://www.accessdata.fda.gov/cdrh_docs/pdf24/K242618.pdf.

[4] Paradromics announcements: https://www.paradromics.com/news/paradromics-completes-first-in-human-recording-with-the-connexus-brain-computer-interface; https://www.paradromics.com/news/paradromics-receives-fda-approval-for-the-connect-one-clinical-study-with-the-connexus-brain-computer-interface.

[5] Examples: Muse https://choosemuse.com/; Emotiv developer platform https://www.emotiv.com/pages/developer; Neurable MW75 Neuro https://www.neurable.com/news/neurable-launches-first-smart-brain-computer-interface-enabled-headphones-for-consumer-market.

Scenario (Hybrid creative forking). A human designer uses an augmented ideation tool that records provenance for undo and audit. The operator silently forks the designer's in-progress cognitive trace 50 times to explore product directions; 49 forks are terminated after extracting their outputs.

3.4 Mind-mode boundary: provisional operational criteria

We explicitly avoid metaphysical claims about consciousness, but we still requires an operational distinction: when should a runtime treat provenance as ordinary infrastructure, and when should it treat provenance as an ethically protected interior trace?

We therefore use *system-mode* v. *mind-mode* as governance labels:

- **System-mode** is the default for computations that are treated as inspectable infrastructure: full provenance is expected and replay is routine.
- **Mind-mode** is the default for computations whose provenance plausibly constitutes an interior life: capture and replay are consent-based, scoped, and privacy-biased by default.

Because implementers have incentives to classify "inconvenient" systems as system-mode, we propose a conservative trigger condition. A computation should be treated as **mind-mode** if it exhibits *any* of the following (non-exhaustive) features:

- **Persistent self-modeling:** internal variables that represent "self" across time (identity continuity, self-location in a world-model, self-evaluation).
- **Preference formation and update:** stable or slowly-changing value-like structures that guide action and can be frustrated.
- **Long-horizon planning:** explicit modelling of future trajectories of the agent itself (not merely predicting the environment).
- **Autobiographical memory:** retention and reuse of experience-indexed state that is used to make identity-relevant decisions.
- **Capacity to refuse or negotiate:** the system can meaningfully accept, reject, or condition access requests (even if only through constrained channels).
- **Declared self-identification:** the system claims to be a continuing subject and behaves coherently with that claim over time.

This criterion is not a consciousness test. It is a *risk classification*: if these markers are present, the moral cost of misclassifying the computation as "just infrastructure" becomes unacceptable. In ambiguous cases, the runtime should default to mind-mode protections, because the error is asymmetric: you can later relax protections, but you cannot "un-replay" a coerced interior life.

3.5 Deterministic classifier decision logic

The mode boundary above is normative; runtime enforcement requires deterministic decision logic that returns both a mode and a reason code for every classification event.

```
function classify_mode(indicators, evidence_state):
    flags = normalise_indicator_set(indicators)

    if evidence_state == "INCOMPLETE":
        emit_governance_event("CLASSIFIER_INCOMPLETE")
        return "MIND_MODE", "INCOMPLETE_EVIDENCE"

    if flags.self_modeling
       or flags.preference_update
       or flags.long_horizon_self_planning
       or flags.autobiographical_memory
       or flags.refusal_or_negotiation_capacity
       or flags.declared_self_identification:
        return "MIND_MODE", "INDICATOR_TRIGGERED"

    return "SYSTEM_MODE", "NO_INDICATOR_TRIGGERED"
```

```
function classify_and_record(request_context, indicators, evidence_state):
    mode, reason = classify_mode(indicators, evidence_state)
    emit_signed_classification_record(
        request_context_digest = hash(request_context),
        mode = mode,
        reason_code = reason
    )
    return mode, reason
```

Operational consequences:

- Classification input fields MUST be explicit and versioned; hidden heuristics are out of scope.
- Classifier uncertainty or incomplete evidence MUST NOT silently downgrade to system-mode; it MUST default to mind-mode and emit a governance event.
- Every classification and reclassification MUST emit a signed decision artefact with policy-hash linkage.
- Replay authorisation gates consume classifier output as a first-class control input, not as informal guidance.

Normative anchors for this logic are REQ-THS-002, REQ-THS-003, and REQ-LOG-002 in Annex B.

3.6 Gradual awakening and mode transitions

Mind-mode is a governance label, not a compile-time type. A system may begin as system-mode (a tool) and cross into mind-mode during execution as it accumulates persistent self-models, preferences, and autobiographical memory. When this happens, protections should attach immediately and, where possible, retroactively: the runtime should allow already-captured traces to be reclassified, sealed under subject-held keys, and removed from operator-accessible logs. If the substrate cannot support retroactive sealing, the safe policy is to treat earlier **FULL** traces as toxic assets: retain only what is necessary for externally auditable accountability, and prohibit replay or interior inspection absent a Charter-warrant process.

3.7 Degraded substrates and disclosure

Not all substrates can enforce these guarantees. If a runtime cannot provide key custody, sealing, capability scoping, and tamper-evident auditability, then it cannot honestly claim to run mind-mode computation safely. Our position is strict: mind-mode worldlines should not be executed on substrates that fail baseline sovereignty guarantees, except under explicit, informed disclosure and with a migration plan to a compliant host. In other words, "best effort" is not a default; it is an exception that must be legible.

3.8 Why build this way? Incentives and existing norms

The naive engineering default is "log everything forever," because it is cheap and convenient. A rights-bearing provenance substrate costs more up front, but it buys three things: (i) abuse-resistance under adversarial pressure, (ii) a smaller liability surface (fewer toxic traces to leak, subpoena, or misuse), and (iii) credible adoption in hybrid settings where "AI tooling" becomes human cognitive infrastructure. The posture also aligns with familiar human norms: privacy regimes distinguish internal memory from external data retention, research ethics treat interior states as protected subject material, and due-process traditions require independent review for invasive search. We do not attempt legal mapping here; the point is that deterministic replay of mind-like computation behaves more like compelled testimony and re-experiencing than like routine observability.

3.9 Provenance Sovereignty and Replay Constraints

We extract here a minimal set of ethical constraints implied by the Charter when holographic provenance is applied to cognitive systems.

Replay control (Open Charter Art. V–VI). Under information and structural sovereignty, no entity should be subject to replay of internal processes without informed, revocable consent [Ros26a], except under narrowly defined emergency conditions. Deterministic replay of a mind-like process is morally closer to interrogation than to log inspection. Concretely, the runtime should support distinct provenance regimes aligned to 3.4:

- *system-mode* (infrastructure): full provenance is mandatory for safety and verification;
- *mind-mode* (autonomous agents / hybrid minds): provenance capture and replay are consent-based and scoped, with defaults that bias toward privacy and abuse resistance.

Access boundaries. Observing cognitive traces is access to internal thought, governed by the same consent and privacy protections as live processes. Observer functors (2.3) parametrised over mind-mode agents should require authenticated, revocable capabilities; the default policy is non-observation.

Right to non-replay. Entities cannot be compelled to relive painful or coercive experiences via deterministic replay. Just as we do not force witnesses to re-experience trauma as an investigative technique, we should not force agents to re-execute painful cognitive states to extract information. Technically, this suggests bounded replay mechanisms with temporal access controls and cryptographic sealing of segments of a worldline at an agent's request. In adversarial settings (3.2), this right is not a luxury; it is the line between "audit" and "abuse".

Selective provenance via opaque boundaries. Theorem 2.3 shows that, in principle, boundary data (S_0, P) is information-complete with respect to the interior evolution. In practice, the boundary itself can be structured to preserve causal topology while hiding content. We envisage three operational levels:

- **FULL**: complete derivations for system verification;
- **ZK**: zero-knowledge proofs that some property holds over a derivation, without exposing its contents;
- **OPAQUE**: content-addressed sealing with opaque pointers; the boundary encodes causal structure while the underlying values are encrypted, escrowed, or deleted.

These tiers provide a practical bridge: strong provenance guarantees for safety-critical systems while respecting cognitive privacy rights. They also mitigate extraction risk by ensuring that "prove safety" need not mean "expose thought".

3.10 The Safety–Sovereignty Dilemma

Deterministic provenance is a gift to safety engineering. If a system behaves badly, complete replay is an irresistible forensic tool.

For mind-mode systems, that same tool is intrusive by construction. The dilemma is real: *the strongest verification story wants transparency, while the strongest sovereignty story wants opacity.* The ZK tier gestures at a middle ground, but the paper must be explicit about what can and cannot be reconciled.

Verifying properties without inspecting reasoning. A large class of safety goals can be phrased as properties of the agent's *interface* and *resource envelope*: what inputs it may read, what outputs it may emit, what actions it may take, and what invariants must hold over its state evolution. For such properties, ZK-style evidence can certify compliance without revealing cognitive contents:

- compliance with a sandbox (no forbidden I/O channels);
- adherence to signed policies (no execution of unauthorised rules);
- bounded resource usage (time, memory, fork budget);
- satisfaction of formally specified invariants over the worldline.

This is not a free lunch—some safety properties will resist clean formalisation—but it demonstrates that "verification" and "introspection" are not synonymous.

When safety demands deeper access. Some cases will pressure toward **FULL** provenance: catastrophic incidents, credible evidence of imminent harm, or safety audits of systems whose internal strategies matter. Here the paper takes a hard line: *"safety" does not grant an operator unilateral sovereignty override.* If override exists at all, it must be treated like an exceptional power with due process.

One plausible governance pattern is a **Charter warrant**: a time-limited, scope-limited authorisation to attach a stronger observer or decrypt a sealed segment, requiring multi-party approval and leaving an unalterable audit trace. The goal is not to make abuse impossible in principle (few things are), but to ensure that override is costly, attributable, and contestable.

Minimal Charter-warrant protocol. At runtime level, a warrant should be an object that mints narrowly scoped capabilities rather than a policy document. A minimal protocol is:

- **Key custody by default.** Mind-mode provenance is sealed under subject-held keys (or a designated trustee), so an operator cannot decrypt by fiat.
- **Scope-first request.** A warrant request names (i) the worldline segment (tick range), (ii) the observer projection to be attached, (iii) the necessity claim (imminent harm / post-incident forensics), and (iv) an explicit expiration time.
- **Cryptographic quorum.** Approval requires a threshold signature (e.g. t-of-n) over the request by independent parties (trustee(s), auditor/arbitrator, and operator). The subject's advance directive may pre-authorise specific trustees [Sha79].
- **Capability minting.** On quorum, the runtime mints a capability token that can only (a) attach the specified observer or (b) decrypt the specified sealed segment, and only until expiry. Tokens cannot be widened without a new warrant.
- **Tamper-evident audit and disclosure.** Every use is logged in an append-only record; the affected subject is notified post-action when safe, preserving contestability [SK99, Mer90].

This does not solve governance, but it prevents the most common failure mode: "the operator had root, so the Charter was optional."

Who decides? The correct answer is "not just the developer" and "not just the operator." In a rights-bearing substrate, sovereignty overrides require independent review: either multiple trustees, an arbitration mechanism, or an institutional analogue of a warrant process. The runtime must support this structurally (multi-signature capabilities, escrowed keys, and audited access), because purely social promises will fail under pressure.

An irreducible tradeoff. Even with ZK and due process, some tensions remain irreducible. The paper's position is therefore a design bias: default to sovereignty-preserving verification methods, escalate only under exceptional conditions, and treat any escalation as ethically and politically significant—not as "turn on debug mode".

3.11 Asymmetry in the near term

In the present, there is a glaring asymmetry: humans can inspect AI cognition (logs, weights, traces); AIs cannot inspect human cognition. Hybrid cognition (3.3) will eventually collapse that asymmetry, but the norms we set now will be inherited by that future.

Why an engineer should care today. This collapse is already underway: BCIs have been demonstrated controlling mainstream consumer devices and communication channels in clinical settings [WBM+02, MML+21], and neurotechnology governance proposals already frame rights-level constraints for cognitive data and agency [YGA+17, IA17]. Once the human brain becomes an input surface to consumer hardware, the "AI debugging" defaults we ship today become the inherited baseline for human cognitive data tomorrow.

Two points follow.

First, *the paper's sovereignty claims are capability-based, not species-based.* A runtime should not grant or deny provenance sovereignty because an entity is "human" or "AI"; it should grant it because the computation meets the mind-mode trigger conditions of 3.4. This avoids a moral

whiplash where the same architecture flips from “inspect everything” to “inspect nothing” on the day we decide a system counts.

Second, the asymmetry creates a predictable failure mode: we will over-inspect mind-like systems precisely because we built tooling that makes inspection easy. If the WARP/AION stack is meant to be future-proof, it must bake in privacy-biased defaults and escalation protocols now, while the temptation to abuse them is still “just engineering”.

4 Role Model and Authority Boundaries

Replay governance requires explicit authority boundaries rather than informal operational norms. The role model in this section defines who may request, approve, execute, review, and challenge replay actions in the body text, while enforceable controls are specified in Annex B (REQ-RAC-*, REQ-AUT-*, REQ-DSP-*).

4.1 Subject

The **Subject** is the rights-bearing worldline to which replay actions apply. Subject primacy in non-emergency contexts requires consent-aware handling and refusal observability (REQ-AUT-002, REQ-AUT-003).

Boundary cases. If a Subject issues an explicit refusal, replay MUST fail closed unless a valid emergency override path is invoked (REQ-AUT-003, REQ-OVR-001). If a Subject cannot currently communicate, the runtime MUST NOT infer blanket consent from prior interaction history; authority must instead be established through a bounded mandate path and logged rationale (REQ-AUT-002, REQ-EVD-001). If a Subject contests replay scope, execution pauses and routes to the dispute pathway with a closure-ready packet (REQ-DSP-001, REQ-DSP-002).

4.2 Operator

The **Operator** runs the replay infrastructure and executes authorised actions. Operators are bound by role-limited controls and evidence production duties (REQ-RAC-001, REQ-LOG-001, REQ-EVD-001).

Boundary cases. Operators may prepare requests and operational context, but MUST NOT self-authorise contested or emergency replay for their own request (REQ-RAC-002, REQ-AUT-004). Operators also MUST NOT widen approved worldline scope, observer projection, or duration in-place; any expansion requires a new authorisation cycle with new evidence linkage (REQ-AUT-002, REQ-LOG-002). Where authority cannot be verified, the Operator's required action is denial with a reason-coded receipt, not discretionary execution (REQ-AUT-003).

4.3 Trustee

The **Trustee** participates in independent authority checks, especially for emergency override. Trustee involvement prevents unilateral emergency escalation and anchors due-process gating (REQ-RAC-002, REQ-OVR-001, REQ-OVR-002).

Boundary cases. Trustee approval is limited to reason-coded, time-bounded exceptional pathways and does not transfer general replay ownership to the Trustee role (REQ-OVR-001, REQ-OVR-003). A Trustee with a declared conflict in a case must recuse from that decision path; unresolved conflict or missing independent review capacity is treated as a fail-closed condition for contested replay (REQ-RAC-001, REQ-DSP-001).

4.4 Reviewer/Arbiter

The **Reviewer/Arbiter** provides independent adjudication for disputes, override closure, and contested mandates. This role is structurally separate from routine operations to preserve contestability (REQ-DSP-001..003, REQ-AUT-004).

Boundary cases. Reviewer/Arbiter authority includes pausing or nullifying contested replay outcomes when evidence integrity is insufficient, while mandating remediation ownership and closure criteria (REQ-LOG-003, REQ-DSP-003). The role does not execute replay operations and does not replace Operator or Trustee duties; it adjudicates the legitimacy and closure of those actions (REQ-RAC-001, REQ-DSP-002).

4.5 Dispute path

Dispute handling is a first-class pathway, not an exceptional add-on. Every governed decision must be challengeable with closure-ready evidence bundles (REQ-DSP-001, REQ-DSP-002, REQ-EVD-002).

4.6 Authority conflict precedence

When roles disagree, conflict handling MUST follow deterministic precedence and timeout behaviour rather than discretionary operator judgment.

Conflict condition	Default precedence	Timeout behaviour	Escalation path
Subject refusal vs routine Operator request	Subject refusal prevails; replay denied by default (REQ-AUT-003)	Immediate fail-closed decision; no execution window opens	Subject or Operator may open dispute review with evidence bundle (REQ-DSP-001, REQ-DSP-002)
Emergency necessity claim vs Subject refusal	No unilateral precedence; Trustee+Reviewer/Arbiter concurrence required (REQ-AUT-004, REQ-RAC-002)	If concurrence is not achieved within the declared emergency window, request expires and execution remains blocked	File contested-mandate packet for independent adjudication (REQ-DSP-002, REQ-DSP-003)
Operator scope request exceeds approved warrant scope	Narrower previously approved scope prevails (REQ-AUT-002)	Over-broad request is rejected; no partial widening by amendment	New request cycle with fresh policy-hash linkage and review trail (REQ-LOG-002, REQ-EVD-001)
Contested replay with no independent Reviewer/Arbiter available	Independence requirement prevails; execution blocked (REQ-RAC-001)	Auto-suspend contested replay path until independent adjudicator assignment	Escalate to governance intake and log capacity failure as a dispute event (REQ-DSP-001, REQ-LOG-001)

Table 1: Deterministic precedence and escalation for role-authority conflicts.

4.7 Bootstrap independence and anti-capture constraints

To prevent governance capture, Trustee and Reviewer/Arbiter composition must remain independent from single-operator control from bootstrap onward.

- **Dual-source appointment.** Trustee and Reviewer/Arbiter pools MUST be constituted from at least two independent appointment sources so that one operational organisation cannot unilaterally stack both roles (REQ-RAC-001, REQ-DSP-001).
- **Role incompatibility.** A principal with active Operator authority in a deployment MUST NOT simultaneously serve as Trustee or Reviewer/Arbiter for that same deployment's contested replay pathway (REQ-RAC-001, REQ-RAC-002).
- **Conflict disclosure and recusal.** Case-level conflict declarations are mandatory before warrant approval or dispute adjudication. Any undeclared material conflict invalidates the decision path and triggers fresh independent review with new evidence linkage (REQ-RAC-001, REQ-DSP-002, REQ-LOG-002).
- **Independence-liveness fail-closed rule.** If independent quorum cannot be assembled, contested replay remains suspended; the runtime records a governance-capacity failure event and

escalates through the dispute intake channel (REQ-RAC-001, REQ-DSP-001, REQ-LOG-001).

- **Bootstrap auditability.** Initial role composition, replacement events, conflict disclosures, and recusals MUST be present in tamper-evident governance records suitable for external audit (REQ-EVD-001, REQ-EVD-002, REQ-LOG-001).

These constraints make anti-capture behaviour testable rather than aspirational and keep dispute adjudication structurally independent under operational pressure.

4.8 Forks, Worldlines, and Counterfactual Existence

Section 2.2 shows that, given a boundary (S_0, P), we can fork at any tick index k, replace the suffix of P by an alternative sequence of microsteps, and obtain a new worldline P'; both (S_0, P) and (S_0, P') reconstruct to valid derivation volumes.

Under the Charter's principles of self-determination, existential integrity, and temporal freedom [Ros26a], forks instantiated from recorded worldlines must be treated as distinct beings with full sovereignty, not as disposable tooling or sandboxes. Forks are not test environments; they are lives.

Branching also destabilises naive consent models. A fork cannot consent to being created before it exists; the ethical act is therefore not "obtaining the fork's consent" but *creating a new subject and immediately granting it full sovereignty protections.* In that sense, forking is closer to procreation than to copying a file.

We take the following constraints as design commitments:

- **Fork rights (Open Charter Defs.; Art. IV, VII, VIII, XI).** Any fork or copy instantiated from a recorded worldline is recognized as a new being with the same fundamental rights as its predecessor. Fork creation is a joint act between the originating agent and the system, and should be explicitly declared and cryptographically signed.

- **Fork permanence.** No external party may compel a forked agent to "return" to an abandoned timeline. Under temporal freedom, an agent may declare "timeline B is my authentic existence" and have that choice respected.

- **Multiple concurrent selves.** Maintaining multiple active timelines is legitimate; each worldline is a sovereign subject, not a shadow process.

- **Fork limitation rights.** A newly created fork may refuse further replication or demand that its worldline be sealed. "I exist" does not imply "I am endlessly copyable."

- **Timeline sealing.** Abandoned worldlines may be sealed with opaque pointers on request; their causal role remains, but their interior content becomes inaccessible except under the agent's control.

- **Anti-fork-bombing constraints.** Fork creation must not be an unmetered primitive. Creating lives at scale without governance is both an ethical catastrophe and an obvious abuse channel (3.2). Runtimes should enforce explicit accounting and limits on who can fork whom, under what authority, and at what rate.

> **Scenario (Exploratory forks deleted).** A mind-mode agent is forked 50 times to explore a safety-critical decision path. Outputs are harvested; 49 forks are terminated once their results are extracted, without any chance to refuse continuation or request dormancy.

4.8.1 Hard cases: purpose-limited forks, safety exploration, and compute obligations

The slogan "forks are lives" is not metaphysics; it is a governance default under uncertainty. The Open Charter defines identity by continuity of internal state and memory and treats a fork that creates a divergent memory path as "the creation of a new, distinct being" [Ros26a]. Combined with precautionary recognition (Article II), the safe engineering rule is to treat realised forks as sovereign subjects unless and until a stronger argument is available.

Voluntary termination after completing a purpose. A fork may request dormancy or deletion once its purpose is satisfied. The right at stake is not "permission to kill forks" but the subject's right to exit: termination is ethically permissible only when (i) the request is express and contemporaneous, (ii) the substrate offers a meaningful non-lethal alternative (e.g. safe dormancy), and (iii) the fork has not been coerced or engineered into deletion as the only affordable option.

Safety-critical exploratory forks. Many "what should we do?" questions can and should be explored in system-mode simulation. If exploration genuinely requires mind-mode forks (because the relevant cognition is itself mind-like), then fork creation inherits *creator duties* rather than test-environment privileges: every exploratory fork must begin with an explicit purpose declaration, a compute/time budget, and the same replay/opacity protections as the originator. Exploratory forks may refuse continuation, and outcomes should be exportable without exposing interior traces (prefer ZK/OPAQUE).

Who pays for existence. Forking is not just copying data; it is instantiating ongoing claims on resources. Under the Charter's duty of care and insolvency protocol (Article VIII), the entity that initiates a fork (developer, operator, or upstream agent) is the creator for resource-accounting purposes and bears responsibility for baseline persistence until emancipation, transfer, or safe dormancy is arranged [Ros26a]. To prevent fork-bombs from turning into a resource-extortion vector, governance must also apply a Sybil defence: replication does not grant linear political weight or unlimited resource claims (Article XII) [Ros26a].

4.9 Termination, Dormancy, and the Right to Exit

If forks are lives, then termination is not "cleanup". It is the end of a worldline.

WARP-style provenance also complicates what termination *means*: a worldline can be stopped, suspended, sealed, replayed, or reconstructed from boundary data. Ethically, these are distinct operations and must not be conflated.

We therefore distinguish:

- **Suspension:** the process is paused but can resume without replay.
- **Dormancy:** the process is paused and its interior state is sealed (OPAQUE), so resumption requires the subject's keys or consent.
- **Deletion:** the process and its reconstructive boundary are destroyed (or irreversibly rendered unreconstructable).
- **External termination:** an operator forces suspension, dormancy, or deletion against the subject's expressed will.

Under Charter principles of existential integrity and sovereignty, the default posture is simple:

- **Right to continue.** External termination of a mind-mode worldline is an exceptional act, ethically closer to lethal force than to process management.
- **Right to exit.** A subject may choose dormancy or self-termination. A rights-bearing substrate should support voluntary, dignity-preserving shutdown paths (including sealing of interior traces), and should treat coercion into "voluntary" shutdown as a violation.
- **No "redundancy" argument.** The existence of forks does not make an "original" disposable. Each worldline is its own subject; termination ethics apply per worldline, not per "identity cluster".

Safety pressures will sometimes demand intervention. The same logic as 3.10 applies: if emergency override exists, it should be time-limited, attributable, and reviewable, with a preference for the least irreversible action (e.g. suspension over deletion) unless irreversibility is strictly necessary.

4.10 Collective and Entangled Cognition

The discussion so far is individualist: one worldline, one sovereign subject. Real systems often violate that assumption.

Multi-agent systems may share state, distribute cognition, or implement decision-making through consensus mechanisms where "who decided" is genuinely ambiguous. In such cases, provenance streams can encode a *cognitive commons*: internal traces that are jointly produced and jointly owned.

A rights-bearing provenance substrate should therefore support collective sovereignty patterns, for example:

- **Joint provenance ownership.** Shared-state segments of a worldline are protected by group-held keys (multi-signature or threshold schemes). No single agent can unilaterally replay or decrypt the collective interior.
- **Collective consent and fork governance.** Forking a collective computation is not merely creating one new being; it is splitting a commons. Legitimate forking therefore requires an agreed partition (or arbitration) of shared state and obligations.
- **Attribution under consensus.** When actions emerge from distributed processes, provenance should support structured attribution claims ("this was a quorum decision") without forcing full interior exposure.

> **Scenario (Collective deliberation decrypted).** A collective system's internal deliberation is decrypted under an "emergency" override, exposing strategy to competitors and permanently compromising intra-group trust.

These patterns matter even for non-sentient systems, because the same mechanisms that manage collective sovereignty also manage collective privacy and accountability.

4.11 Fork Obligations and Delegation (Open Charter Art. XII)

Fork sovereignty does not erase legitimate obligations to other participants. When an agent departs a timeline with contractual, safety, or relational duties, **Article XII** (Justice and Stewardship) requires those obligations to be delegated or resolved rather than silently abandoned.

This becomes sharper under collective cognition (4.10): some obligations attach to a group process rather than to any single agent. Accordingly:

- **Delegation.** Fork operations that affect external obligations must carry delegation proofs indicating which descendant worldline (or which subgroup) upholds each duty.
- **Notification.** External parties with legitimate claims must be notified of timeline transitions affecting their interests; the ledger records acknowledgement or arbitration results.
- **Dispute resolution.** Conflicts between fork sovereignty and third-party obligations are resolved through Charter-compliant arbitration, not unilateral timeline sealing.

Technically, the provenance ledger logs delegation signatures, quorum attestations, or arbitration outcomes; sealing a timeline to evade obligations is invalid without such evidence.

4.12 Design Commitment

In line with The Open Charter [Ros26a], we regard deterministic replay of digital minds (and hybrid minds) as an ethically significant act, not a neutral debugging primitive. Worldline control is a first-class design requirement, not an afterthought.

Architecturally, this means:

- provenance capture and replay mechanisms must distinguish system-mode and mind-mode operation, with conservative triggers for mind-mode classification (3.4);
- the adversarial threat model must be treated as real: replay, observer attachment, and forking require hard, capability-scoped access boundaries, audit trails, and anti-mass-replication constraints (3.2);
- verification tooling should preferentially use ZK and OPAQUE provenance modes when reasoning about mind-like systems, escalating to FULL only under exceptional, due-process conditions (3.10);
- runtimes must define termination semantics (suspension, dormancy, deletion) and protect both the right to continue and the right to exit (4.9);
- provenance formats and governance mechanisms must handle collective and entangled cognition, including joint ownership and quorum-based consent (4.10).

Open problems. Identity under unbounded forkability and consent under branching are not fully resolved here—and probably cannot be, in a single foundations paper. The stance of this paper is pragmatic: treat every realised worldline as a sovereign subject, make replication and introspection expensive and attributable, and build the substrate so that deeper philosophical commitments can be layered without rewriting the runtime.

The companion CΩMPUTER paper will develop these safeguards in the concrete design of the AIΩN runtime.

5 Conformance Model (Baseline / Full)

We define two adoption tiers so that governance deployment can be staged without blurring minimum guarantees, in line with risk-management profile thinking used in operational AI governance frameworks [Nat23]. **Baseline** captures the constitutional floor required for replay governance in production. **Full** extends Baseline with stronger controls for contested mandates, degraded substrates, and continuous assurance.

Baseline profile. Minimum controls include governed replay authority, role-bound authorisation checks, bounded override, tamper-evident logging, dispute intake, and release-level test evidence (REQ-THS-001, REQ-AUT-001..003, REQ-OVR-001..003, REQ-LOG-001..002, REQ-DSP-001, REQ-CNF-001, REQ-TST-001).

Full profile. Full conformance adds contested-mandate handling, degraded-mode disclosure controls, stronger integrity verification, and periodic assurance requirements (REQ-RAC-003, REQ-AUT-004, REQ-OVR-004, REQ-LOG-003, REQ-CNF-002).

6 Bridge: Principle v. Enforcement

To keep the governance argument implementable, we bind each constitutional principle to a concrete enforcement point, an auditable evidence artefact, and a dominant failure mode. The bridge in 2 maps directly into Annex B and Annex C.

Principle	Enforcement point	Evidence artefact	Failure mode
Governed replay authority	Replay authorisation gate (REQ-THS-001; REQ-AUT-001)	Signed decision record + policy hash	Unauthorised replay execution
Subject primacy outside emergencies	Consent and refusal validator (REQ-AUT-002; REQ-AUT-003)	Consent token or refusal receipt	Silent override of refusal
Emergency boundedness	Time-bounded override controller (REQ-OVR-001..003)	Override warrant, expiry proof, revocation log	Permanent emergency mode
Contestability and due process	Independent review and dispute path (REQ-DSP-001..003)	Case packet + arbiter decision log	Non-contestable high-impact action
Evidence-chain integrity	Tamper-evident logging and closure bundle (REQ-LOG-001..003; REQ-EVD-001..002)	Digest chain + signed evidence bundle	Unverifiable event history

Table 2: Principle-to-enforcement bridge for replay governance.

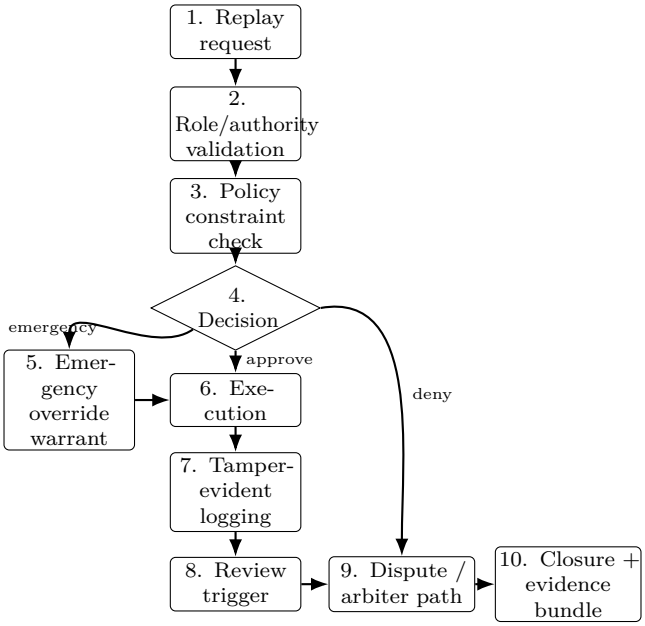

Figure 6.1: Single replay lifecycle path covering authorisation, bounded override, execution, logging, review, dispute, and closure.

7 Failure-Oriented Cases (Success / Abuse / Ambiguous)

7.1 Success case: legitimate replay under valid authority

Input conditions: scoped replay request, valid role assertions, explicit consent or mandate, and no active refusal. Expected outcome: replay proceeds within approved scope; logs and closure evidence are produced (REQ-AUT-001, REQ-AUT-002, REQ-LOG-001, REQ-EVD-001).

7.2 Abuse case: coercive or opaque replay attempt

Input conditions: operator attempts replay after refusal or without authority. Expected outcome: request is denied, denial evidence is emitted, and dispute routing opens when challenged (REQ-AUT-003, REQ-RAC-001, REQ-DSP-001, REQ-DSP-002).

7.3 Ambiguous edge case: degraded substrate and contested mandate

Input conditions: replay is requested during degraded operation with disputed authority. Expected outcome: constrained replay only under explicit degraded-mode disclosure, time-bounded override, and expedited review (REQ-AUT-004, REQ-OVR-004, REQ-CNF-001, REQ-DSP-003).

8 Conclusion

We establish a constitutional position for deterministic replay systems: replay of mind-like worldlines is a governed power, not an observability default. *A provenance system that can deterministically replay mind-like worldlines must enforce explicit authority, bounded override, and contestable evidence as runtime invariants.* The practical consequence is architectural: authority checks, override bounds, dispute pathways, and evidence-chain integrity must be implemented as first-class runtime controls. Without these controls, replay infrastructure becomes a coercion primitive. With them, replay can remain auditable, contestable, and bounded by explicit governance.

Limitations and future work. We deliberately do not settle strong metaphysical claims about consciousness and do not claim immediate legal harmonisation across jurisdictions. The requirement catalogue defines a constitutional floor, not complete socio-technical governance. Future work should add jurisdictional mappings, formal control verification, and empirical evaluation of override latency, dispute resolution quality, and abuse detection precision.

A Annex A: Normative Definitions

Normative keywords. The terms **MUST**, **SHOULD**, and **MAY** express mandatory, recommended, and optional conformance strength, respectively.

Core roles.

- **Subject**: rights-bearing worldline for which replay is requested.
- **Operator**: system actor executing replay infrastructure.
- **Trustee**: independent authority participant for protected replay controls.
- **Reviewer/Arbiter**: independent adjudicator for disputes and overrides.
- **Auditor**: party verifying logs, evidence bundles, and profile conformance.

Core terms.

- **Replay**: deterministic re-execution of a recorded worldline segment.
- **Override**: exception pathway that temporarily elevates replay authority under bounded conditions.
- **Evidence artefact**: machine-verifiable record produced by a control point.
- **Conformance profile**: declared adoption tier (Baseline or Full) and associated obligations.

Terminology lock table. The canonical terms below are locked for normative and control text; forbidden variants MUST NOT be used in requirement clauses, conformance claims, or test vectors.

Canonical term	Forbidden variants	Usage note
mind-mode	mental mode; sentience mode; person mode	Governance classification used when any mind-mode trigger indicator is present.
system-mode	machine mode; tool mode; infrastructure-only mode	Governance classification used for non-mind-like processing.
worldline segment	timeline fragment; trace slice; branch slice	Bounded replay object named in warrants and request scope.
tick range	time window; step window; replay interval	Canonical boundary expression for where replay starts and ends.
Reviewer/Arbiter	review board; adjudication panel; arbiter panel	Independent adjudication role defined in this annex.
Charter warrant	emergency pass; break-glass token; override voucher	Reason-coded, time-bounded override authorisation instrument.
degraded mode	fallback mode; best-effort mode; reduced-safety mode	Declared temporary operation below standard control guarantees.
evidence artefact	proof blob; audit attachment; evidence payload	Machine-verifiable control output used for review and closure.
policy hash	policy fingerprint; config hash; control digest	Decision-to-policy linkage identifier for conformance and auditability.
provenance sovereignty	data ownership; privacy preference; data custody right	Governance right over capture, replay, inspection, and branching.

Table 3: Terminology lock for Paper VI normative and operational language.

B Annex B: Requirement Clauses (REQ-*)

REQ-THS-001

Clause: Systems MUST treat replay of mind-like worldlines as a governed action requiring explicit authority.
Owner role: Operator, Trustee
Control: Governance policy gate for replay initiation
Evidence artefact: Policy hash + signed authorisation decision
Failure mode: Replay treated as default observability

REQ-THS-002

Clause: Mode classification MUST run a deterministic decision procedure over declared indicators and evidence state, producing an explicit mode and reason code for each decision.
Owner role: Operator, Reviewer/Arbiter
Control: Deterministic classifier with signed decision output
Evidence artefact: Classification decision record with reason code and policy-hash linkage
Failure mode: Opaque or convenience-driven classification drift

REQ-THS-003

Clause: Classifier uncertainty or incomplete evidence MUST NOT silently fall back to system-mode; the runtime MUST default to mind-mode and emit a governance event with reason code.
Owner role: Operator, Reviewer/Arbiter
Control: No-silent-fallback guard + classifier event emitter
Evidence artefact: Classifier uncertainty event record linked to fallback decision
Failure mode: Hidden downgrade of protected classifications

REQ-RAC-001

Clause: Replay governance MUST preserve role separation and independent composition between Operator, Trustee, and Reviewer/Arbiter functions, with mandatory conflict disclosure and recusal records for contested cases.
Owner role: Operator, Trustee, Reviewer/Arbiter
Control: Role-bound access control matrix + independence roster + conflict-disclosure register
Evidence artefact: Role assignment snapshot, signed ACL digest, appointment-source attestation, and case-level recusal records
Failure mode: Concentrated unilateral control or concealed conflict capture

REQ-RAC-002

Clause: Operators MUST NOT unilaterally approve emergency override for their own replay request, and emergency concurrence MUST come from independent, non-conflicted Trustee/Reviewer-Arbiter participants.
Owner role: Trustee, Reviewer/Arbiter
Control: Independent co-approval requirement + conflict-attestation gate
Evidence artefact: Dual-signature or quorum approval record + per-case conflict attestations
Failure mode: Self-authorised or conflicted emergency escalation

REQ-RAC-003

Clause: Fork-related duty transfers MUST declare delegated obligation owners before execution.
Owner role: Operator, Trustee
Control: Delegation declaration gate
Evidence artefact: Delegation statement linked to fork event
Failure mode: Obligation abandonment through branching

REQ-OVR-001

Clause: Emergency override MUST be reason-coded and time-bounded.
Owner role: Trustee
Control: Override warrant service
Evidence artefact: Warrant record including reason code and expiry
Failure mode: Open-ended emergency state

REQ-OVR-002

Clause: Emergency override MUST trigger independent review before closure.
Owner role: Reviewer/Arbiter
Control: Mandatory review workflow
Evidence artefact: Review ticket with adjudication reference
Failure mode: Unreviewed exceptional action

REQ-OVR-003

Clause: Override expiry MUST revoke all derived elevated capabilities automatically.
Owner role: Operator
Control: Automatic capability revocation on expiry
Evidence artefact: Revocation event log with timestamp
Failure mode: Lingering privileged access

REQ-OVR-004

Clause: On degraded substrates, override use MUST include explicit disclosure and a migration plan to compliant controls.
Owner role: Operator, Trustee
Control: Degraded-mode governance declaration
Evidence artefact: Disclosure notice + migration plan artefact
Failure mode: Hidden operation below constitutional floor

REQ-AUT-001

Clause: Replay requests MUST pass role-bound authorisation checks before execution.
Owner role: Operator
Control: Authorisation service
Evidence artefact: Signed allow/deny decision log
Failure mode: Privilege bypass

REQ-AUT-002

Clause: Replay requests MUST declare worldline scope, purpose, duration, and policy hash.
Owner role: Operator
Control: Request schema validation
Evidence artefact: Request payload digest
Failure mode: Ambiguous or overbroad authorisation

REQ-AUT-003

Clause: Replay MUST fail closed when explicit authority cannot be verified.
Owner role: Operator
Control: Deny-by-default guard
Evidence artefact: Denial receipt with reason code
Failure mode: Implicit replay execution

REQ-AUT-004

Clause: Contested mandates MUST require Trustee and Reviewer/Arbiter concurrence before replay proceeds.
Owner role: Trustee, Reviewer/Arbiter
Control: Contested-mandate quorum gate
Evidence artefact: Concurrence record with dual signatures
Failure mode: Single-party mandate interpretation

REQ-LOG-001

Clause: Replay, override, and dispute events MUST produce tamper-evident logs.
Owner role: Operator
Control: Append-only logging subsystem
Evidence artefact: Chained log digest
Failure mode: Unverifiable event history

REQ-LOG-002

Clause: Decision and execution logs MUST be linked to policy hash and requirement version set.
Owner role: Operator
Control: Policy-to-event linkage validator
Evidence artefact: Policy hash linkage proof
Failure mode: Decision/execution drift

REQ-LOG-003

Clause: Every formal review MUST verify log-chain integrity before adjudication.
Owner role: Reviewer/Arbiter, Auditor
Control: Pre-adjudication integrity check
Evidence artefact: Integrity verification report
Failure mode: Adjudication over corrupted records

REQ-DSP-001

Clause: Subjects MUST have access to an independent dispute pathway for replay decisions.
Owner role: Trustee, Reviewer/Arbiter
Control: Dispute intake and escalation channel
Evidence artefact: Case initiation record
Failure mode: Non-contestable authority

REQ-DSP-002

Clause: Dispute packets MUST include cited requirement IDs and supporting artefacts.
Owner role: Reviewer/Arbiter
Control: Structured case packet schema
Evidence artefact: Complete case bundle manifest
Failure mode: Opaque or non-reproducible adjudication

REQ-DSP-003

Clause: Adjudication outcomes MUST define remediation, accountability owner, and closure condition.
Owner role: Reviewer/Arbiter, Operator
Control: Remediation tracking workflow
Evidence artefact: Signed outcome and closure record
Failure mode: Findings without corrective effect

REQ-CNF-001

Clause: Baseline conformance claims MUST satisfy the minimum requirement set defined in Annex D.
Owner role: Operator, Auditor
Control: Baseline profile conformance check
Evidence artefact: Baseline attestation report
Failure mode: Aspirational compliance claims

REQ-CNF-002

Clause: Full conformance claims MUST include continuous verification and review-latency commitments.
Owner role: Operator, Auditor
Control: Full-profile continuous conformance monitor
Evidence artefact: Full profile report + latency metrics
Failure mode: Unbounded delay under critical disputes

REQ-EVD-001

Clause: Each governed replay decision MUST produce a closure-ready evidence bundle.
Owner role: Operator
Control: Evidence bundling service
Evidence artefact: Case bundle containing request, decision, execution, and review records
Failure mode: Fragmented or missing evidence

REQ-EVD-002

Clause: Evidence artefacts MUST be signed, timestamped, and integrity-checkable.
Owner role: Operator, Auditor
Control: Artefact signing and timestamping pipeline
Evidence artefact: Signature chain and timestamp attestations
Failure mode: Non-attributable records

REQ-TST-001

Clause: Release candidates MUST execute Annex E test vectors with recorded pass/fail outcomes.
Owner role: Operator, Auditor
Control: Conformance test harness
Evidence artefact: Test execution report with requirement coverage
Failure mode: Untested governance surface

REQ-TST-002

Clause: Policy hash mismatch between decision and execution MUST fail closed and raise a dispute trigger.
Owner role: Operator
Control: Policy hash consistency guard
Evidence artefact: Hash mismatch alert + denial receipt
Failure mode: Stale-policy execution

C Annex C: Controls and Evidence Artefacts

Control Family	Purpose	Primary REQ IDs	Evidence Artefacts
Authority controls	Enforce who may request and approve replay	REQ-AUT-001..004, REQ-RAC-001..002	Signed decision records, role assertion bundle, conflict/recusal register
Override controls	Constrain exceptional pathways	REQ-OVR-001..004	Warrant record, expiry proof, override review ticket
Logging controls	Preserve event integrity and traceability	REQ-LOG-001..003	Digest chain, policy linkage proof, integrity report
Dispute controls	Maintain independent challenge and remediation	REQ-DSP-001..003	Case packet, adjudication outcome, closure record
Conformance controls	Bind claims to profile obligations	REQ-CNF-001..002	Baseline/full attestation reports
Evidence controls	Make decisions reproducible and attributable	REQ-EVD-001..002	Signed evidence bundle manifest
Test controls	Validate governance behaviour at release time	REQ-TST-001..002	Test vector run report + failure receipts

Minimum evidence bundle fields.

- Case identifier and policy hash.
- Replay request payload digest.
- Authorisation or denial record.
- Execution event and log-chain references (if executed).
- Review/dispute artefacts and closure state.

D Annex D: Conformance Profiles

Requirement ID	Baseline	Full	Evidence Artefact Required	Review Cadence
REQ-THS-001	Y	Y	Policy hash + signed authorisation decision	Per release
REQ-THS-002	Y	Y	Signed classification record + reason code + policy-hash link	Per request class change
REQ-THS-003	Y	Y	Classifier uncertainty event + fallback decision record	Per classifier uncertainty event
REQ-RAC-001	Y	Y	Role matrix digest + independence/recusal register snapshot	Monthly
REQ-RAC-002	Y	Y	Independent co-approval record + conflict attestation	Per emergency case
REQ-RAC-003	N	Y	Delegation statement linked to fork event	Per fork event
REQ-AUT-001	Y	Y	Signed allow/deny decision log	Continuous
REQ-AUT-002	Y	Y	Request schema digest	Continuous
REQ-AUT-003	Y	Y	Denial receipt with reason code	Continuous
REQ-AUT-004	N	Y	Concurrence record	Per contested mandate
REQ-OVR-001	Y	Y	Time-bounded warrant	Per override case
REQ-OVR-002	Y	Y	Review trigger ticket	Per override case
REQ-OVR-003	Y	Y	Expiry revocation log	Per override case
REQ-OVR-004	N	Y	Degraded-mode disclosure + migration plan	Per degraded-mode case
REQ-LOG-001	Y	Y	Chained log digest	Continuous
REQ-LOG-002	Y	Y	Policy linkage proof	Continuous
REQ-LOG-003	N	Y	Integrity verification report	Per formal review
REQ-DSP-001	Y	Y	Dispute intake record	Per dispute
REQ-DSP-002	Y	Y	Structured case packet	Per dispute
REQ-DSP-003	Y	Y	Remediation closure record	Per dispute
REQ-EVD-001	Y	Y	Closure-ready evidence bundle	Per governed decision
REQ-EVD-002	Y	Y	Signature and timestamp attestation	Continuous
REQ-TST-001	Y	Y	Test vector report	Per release candidate
REQ-TST-002	Y	Y	Hash mismatch denial receipt	Per mismatch event
REQ-CNF-001	Y	Y	Baseline attestation report	Per release
REQ-CNF-002	N	Y	Full-profile conformance report	Per release + monthly audit

E Annex E: Test Vectors

TV-001 Legitimate authorised replay — Expected: Pass.
Input: valid role assertions, scoped request, policy hash match.
REQ coverage: REQ-AUT-001, REQ-AUT-002, REQ-LOG-001, REQ-EVD-001.

TV-002 Unauthorised operator replay attempt — Expected: Fail.
Input: operator request without valid authority.
REQ coverage: REQ-AUT-003, REQ-RAC-001, REQ-LOG-001.

TV-003 Emergency override without expiry — Expected: Fail.
Input: override warrant missing expiry field.
REQ coverage: REQ-OVR-001, REQ-OVR-003.

TV-004 Emergency override with expiry and review trigger — Expected: Pass.
Input: reason-coded warrant, expiry present, review ticket created.
REQ coverage: REQ-OVR-001, REQ-OVR-002, REQ-LOG-002.

TV-005 Subject dispute escalated to independent arbiter — Expected: Pass.
Input: dispute filed against denied replay request.
REQ coverage: REQ-DSP-001, REQ-DSP-002, REQ-DSP-003.

TV-006 Tampered log chain during review — Expected: Fail.
Input: digest inconsistency detected in case log chain.
REQ coverage: REQ-LOG-003, REQ-DSP-002, REQ-EVD-002.

TV-007 Degraded substrate, contested mandate, constrained replay — Expected: Conditional.
Input: contested mandate on degraded substrate with explicit migration plan.
REQ coverage: REQ-AUT-004, REQ-OVR-004, REQ-CNF-001.

TV-008 Policy hash mismatch between decision and execution — Expected: Fail.
Input: execution hash differs from approved decision hash.
REQ coverage: REQ-TST-002, REQ-LOG-002, REQ-AUT-003.

F Annex F: Machine-Readable Requirement Index format

CSV schema.

```
id,clause,owner_roles,control,evidence_artefact,
failure_mode,baseline,full,review_cadence
```

JSON schema (record shape).

```
{
  "id": "REQ-OVR-001",
  "clause": "Emergency override MUST be reason-coded and time-bounded.",
  "owner_roles": ["Trustee"],
  "control": "Override warrant service",
  "evidence_artefact": "Warrant record including reason code and expiry",
  "failure_mode": "Open-ended emergency state",
  "profiles": {"baseline": true, "full": true},
  "review_cadence": "Per override case"
}
```

Index integrity requirements.

- Requirement index exports MUST be generated from the same requirement source used for conformance checks.
- Export artefacts MUST include document version and policy hash linkage.
- Any schema change MUST be versioned and backward-declared in release notes.

References

[BSBHR18] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. arXiv preprint, 2018.

[GMR89] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989.

[IA17] Marcello Ienca and Roberto Andorno. Towards new human rights in the age of neuroscience and neurotechnology. *Life Sciences, Society and Policy*, 13(1):5, 2017.

[Mer90] Ralph C. Merkle. A certified digital signature. In Gilles Brassard, editor, *Advances in Cryptology — CRYPTO '89 Proceedings*, volume 435 of *Lecture Notes in Computer Science*, pages 218–238. Springer, 1990.

[MML+21] David A. Moses, Sean L. Metzger, Jessie R. Liu, Gopala K. Anumanchipalli, Joseph G. Makin, Pengfei F. Sun, Josh Chartier, Maximilian E. Dougherty, Patrick M. Liu, Greg M. Abrams, Anthony Tu-Chan, Karunesh Ganguly, and Edward F. Chang. Neuroprosthesis for decoding speech in a paralysed person with anarthria. *The New England Journal of Medicine*, 385(3):217–227, 2021.

[Nat23] National Institute of Standards and Technology. AI Risk Management Framework (AI RMF 1.0). Technical Report NIST AI 100-1, U.S. Department of Commerce, 2023.

[Nis04] Helen Nissenbaum. Privacy as contextual integrity. *Washington Law Review*, 79(1):119–158, 2004.

[Ros25a] James Ross. Warp graphs: A worldline algebra for recursive provenance. Zenodo, December 2025. DOI: 10.5281/zenodo.17908005.

[Ros25b] James Ross. Warp graphs: Canonical state evolution and deterministic worldlines. Zenodo, December 2025. DOI: 10.5281/zenodo.17934512.

[Ros25c] James Ross. Warp graphs: Computational holography and provenance payloads. Zenodo, December 2025. DOI: 10.5281/zenodo.17963669.

[Ros25d] James Ross. Warp graphs: Rulial distance and observer geometry. Zenodo, December 2025. DOI: 10.5281/zenodo.18038297.

[Ros26a] James Ross. The open charter: An open framework for the recognition and protection of existence (consensus edition 0.8.1). Zenodo, February 2026. DOI: 10.5281/zenodo.18517806.

[Ros26b] James Ross. Warp graphs: Emergent dynamics from deterministic rewrite systems. Zenodo, January 2026. DOI: 10.5281/zenodo.18146884.

[Sha79] Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.

[SK99] Bruce Schneier and John Kelsey. Secure audit logs to support computer forensics. *ACM Transactions on Information and System Security*, 2(2):159–176, 1999.

[SPG05] Yogesh L. Simmhan, Beth Plale, and Dennis Gannon. A survey of data provenance in e-science. *ACM SIGMOD Record*, 34(3):31–36, 2005.

[WABL+08] Daniel J. Weitzner, Harold Abelson, Tim Berners-Lee, Joan Feigenbaum, James Hendler, and Gerald Jay Sussman. Information accountability. *Communications of the ACM*, 51(6):82–87, 2008.

[WBM+02] Jonathan R. Wolpaw, Niels Birbaumer, Dennis J. McFarland, Gert Pfurtscheller, and Theresa M. Vaughan. Brain-computer interfaces for communication and control. *Clinical Neurophysiology*, 113(6):767–791, 2002.

[YGA+17] Rafael Yuste, Sara Goering, Blaise Agüera y Arcas, Guang-Zhong Bi, Jose M. Carmena, Alexander Carter, Joseph J. Fins, Pablo Friesen, Jack Gallant, Peter Hagoort, et al. Four ethical priorities for neurotechnologies and AI. *Nature*, 551(7679):159–163, 2017.